

Month 01 Capstone - NYC Taxi Data Product

2026-08-17

Data Engineering Course

NYC Taxi Data Product

Real data -> reliable SQL model -> validated
metrics -> business report

Cel projektu: pokazac, ze potrafisz zbudowac maly, odtwarzalny produkt danych, a nie tylko napisac pojedyncze zapytania SQL.

■ Executive summary

Budujesz lokalny produkt analityczny na publicznych danych NYC TLC Yellow Taxi.

Projekt ma pokazać cztery kompetencje:

1. Rozumiesz odbiorcę i pytania biznesowe.
2. Umiesz zamienić surowe pliki w model raw / silver / gold.
3. Umiesz zdefiniować grain, metryki i walidacje.
4. Umiesz opisać jak projekt odtworzyć, utrzymać i debugować.

Szczegółowe komendy i wymagane pliki są w briefie:

```
02_month_01_capstone_project_instructions.md
```

■ Business context

Odbiorca: zespół city operations / mobility analytics.

Pytania, na które produkt ma odpowiedzieć:

Ktore strefy generują największy popyt i revenue?
W których godzinach występują szczyty popytu?
Jak zachowują się przejazdy z lotnisk?
Czy wyniki są odtwarzalne i wystarczająco zaufane?

To jest projekt o decyzjach biznesowych, nie o samym uruchomieniu DuckDB.

■ Product outcome

Finalny output ma być czytelny dla dwóch osób.

Data consumer:

czyta reports/01_business_summary.md i rozumie najważniejsze wnioski

Data engineer / reviewer:

czyta sql/, docs/ i README.md i potrafi odtworzyć pipeline

Profesjonalny projekt danych łączy wynik biznesowy z techniczną odtwarzalnością.

■ Scope

W zakresie:

```
DuckDB  
SQL  
Parquet + CSV  
raw / silver / gold  
fact + dimension  
data quality checks  
EXPLAIN / performance note  
backfill plan  
incident runbook  
business summary
```

Poza zakresem:

```
Airflow, dbt, Spark, cloud, Python ingestion,  
dashboard BI
```

Month 01 sprawdza fundamenty SQL Data Engineering.

Uzywasz realnych danych NYC TLC.

```
yellow_tripdata_2024-01.parquet  
yellow_tripdata_2024-02.parquet  
yellow_tripdata_2024-03.parquet  
yellow_tripdata_2024-04.parquet  
yellow_tripdata_2024-05.parquet  
yellow_tripdata_2024-06.parquet  
taxi_zone_lookup.csv
```

Zrodlo:

```
https://www.nyc.gov/site/tlc/about/tlc-trip-record-  
-data.page
```

Pliki 2024-01 do 2024-06 oraz lookup CSV zostaly zweryfikowane 2026-08-16.

■ Volume decision

Docelowo pracujesz na 6 miesiacach danych.

```
Dlaczego 6 miesiecy:  
  widac zmiany miesiac do miesiaca  
  performance zaczyna miec znaczenie  
  backfill ma realny sens  
  walidacje row count nie sa tylko formalnoscia
```

Minimalny zakres akceptowalny: 3 miesiace.

Jesli ograniczasz zakres, opisujesz powod w README jako znane ograniczenie.

```
NYC TLC Parquet / CSV
  |
  v
raw views
  |
  v
silver_taxi_trips + rejected_taxi_trips
  |
  v
dim_taxi_zone + fact_taxi_trips
  |
  v
gold_daily_zone_metrics
gold_hourly_demand
gold_airport_metrics
  |
  v
validation checks + business report
```

Ta architektura pokazuje przepływ danych, odpowiedzialność warstw i miejsce kontroli jakości.

■ Why DuckDB

DuckDB jest dobrym narzędziem do lokalnego capstone, bo pozwala pracować bez platformy chmurowej.

Czyta Parquet bez ręcznego importu.
Działa lokalnie na laptopie.
Pozwala uczyć się SQL, modelowania i performance.
Nie ukrywa logiki za frameworkiem.

Kluczowy wzorzec:

```
SELECT *  
FROM read_parquet('data/raw/tlc/yellow/year=2024/month=*/yellow_tripdata_2024-*.parquet');
```

■ Layer responsibilities

```
raw:
  faithful read of source files; no business
  cleanup

silver:
  typed, filtered, explainable accepted trips

rejected:
  records not used in metrics, with reason

fact:
  accepted trip-level event table

dimension:
  zone metadata used for business-readable metrics

gold:
  aggregated tables answering concrete business
  questions
```

Kazda warstwa ma miec jasny powod istnienia.

Minimalny model koncowy:

```
dim_taxi_zone
  one row = one NYC taxi zone

fact_taxi_trips
  one row = one accepted Yellow Taxi trip

gold_daily_zone_metrics
  one row = one day + pickup borough + pickup zone

gold_hourly_demand
  one row = one day + pickup hour

gold_airport_metrics
  one row = one day + airport pickup zone
```

Bez grain nie wiadomo, co oznacza COUNT, SUM ani unikalnosc wyniku.

■ Quality policy

Nie filtrujesz danych po cichu.

Akceptowany rekord musi spełnić minimum:

```
pickup_at is not null  
dropoff_at is not null  
pickup_at < dropoff_at  
trip_distance > 0  
total_amount >= 0
```

Odrzucone rekordy trafiają do `rejected_taxi_trips` z `reject_reason`.

To jest różnica między czyszczeniem danych a kontrolowanym procesem jakości.

■ Gold tables

Gold layer ma odpowiadać na pytania odbiorcy.

```
gold_daily_zone_metrics:  
    gdzie jest revenue i popyt dzień po dniu?  
  
gold_hourly_demand:  
    które godziny mają najwyższy popyt?  
  
gold_airport_metrics:  
    jak lotniska różnią się od reszty miasta?
```

Każda tabela gold musi mieć opisany grain, metryki i znane ograniczenia interpretacji.

■ Required SQL capabilities

Projekt ma naturalnie pokazac fundamenty Month 01.

```
SELECT / WHERE / GROUP BY  
JOIN fact -> dimension  
CTE for readable transformations  
CASE WHEN for reject reasons or categories  
window function for ranking or month-over-month  
comparison  
CREATE OR REPLACE for idempotency  
EXPLAIN for performance review
```

Nie chodzi o uzycie syntaxu dla checkboxa. Kazdy element ma wynikac z problemu.

■ Validation strategy

Walidacje odpowiadaja na pytanie: czy mozemy pokazac metryki odbiorcy?

Minimum:

```
silver has rows  
rejected records have reject_reason  
silver has no negative total_amount  
silver has no non-positive trip_distance  
gold_daily_zone_metrics has unique grain  
fact_taxi_trips joins to dim_taxi_zone as expected  
smoke test confirms main tables exist and contain  
rows
```

Dobre walidacje sa konkretne i maja opisany oczekiwany wynik.

■ Operational thinking

Ten projekt nie ma schedulera, ale ma pokazac, ze rozumiesz jak pipeline dziala operacyjnie.

Dokumentujesz:

```
run metadata design:  
  co zapisalbys o kazdym uruchomieniu  
  
backfill plan:  
  jak przeliczysz poprawiony miesiac danych  
  
incident runbook:  
  jak zbadasz nagly revenue spike
```

To odroznia projekt Data Engineering od folderu z query.

Nie musisz być ekspertem od optymalizacji.

Masz pokazać, że umiesz zadawać dobre pytania:

```
Ktore query czyta najwiecej danych?  
Czy filtrujesz dane przed agregacja gold?  
Czy wybierasz tylko potrzebne kolumny?  
Czy gold tables ograniczaja ponowne liczenie  
wszystkiego od zera?  
Co zmienilbys, gdyby danych bylo 100x wiecej?
```

Użyj EXPLAIN i opisz obserwacje prostym językiem.

■ Deliverable structure

Oddajesz uporządkowany katalog projektu.

```
homework/month_01_capstone/  
├─ sql/      raw, silver, fact, dim, gold,  
checks, smoke test  
├─ docs/     grain, metrics, contract, runs,  
backfill, runbook  
├─ reports/  business summary dla odbiorcy  
├─ README.md reproducible project guide  
└─ .gitignore
```

Nie commitujesz dużych Parquetów, lokalnej bazy DuckDB ani cache.

Oczekiwany zestaw SQL:

```
00_bootstrap.sql
01_raw_sources.sql
02_raw_profiling.sql
03_silver_trips.sql
04_dimensions.sql
05_fact_trips.sql
06_gold_daily_zone_metrics.sql
07_gold_hourly_demand.sql
08_gold_airport_metrics.sql
09_validation_checks.sql
10_performance_review.sql
11_smoke_test.sql
```

To jest pipeline podzielony na odpowiedzialności, nie jeden plik `all_queries.sql`.

Oczekiwane dokumenty:

```
01_problem_statement.md
02_data_sources.md
03_grain_and_model.md
04_metric_definitions.md
05_data_quality_contract.md
06_pipeline_runs_design.md
07_backfill_plan.md
08_incident_runbook.md
09_query_review.md
10_interview_answer.md
```

Dokumentacja nie jest dodatkiem po projekcie. To część produktu danych.

■ Business report

reports/01_business_summary.md ma być napisany dla odbiorcy, nie tylko dla reviewera technicznego.

Powinien zawierać 5-8 insightów, na przykład:

```
top pickup zones by revenue  
top pickup zones by trip count  
strongest demand hours  
airport revenue share  
highest revenue month  
data quality observation  
metric interpretation risk
```

SQL tworzy wynik. Raport tłumaczy, co z niego wynika.

■ Review rubric

Reviewer będzie oceniał trzy obszary.

Reproducibility:

czy da się uruchomić projekt od zera z README

Engineering quality:

czy model, walidacje i SQL są spójne

Product quality:

czy wynik odpowiada na realne pytania i jest czytelny

Dobry projekt broni się jako całość: kod, dane, dokumentacja i raport.

■ Levels of completion

Level 1 - works end-to-end:

raw -> silver -> fact/dim -> gold -> basic checks

Level 2 - professional capstone:

6 months, rejects, validation, EXPLAIN, backfill, runbook, contract

Level 3 - portfolio-ready:

strong README, clear insights, month-over-month analysis, polished docs

Minimalny cel kursu: Level 2.

Ambitny cel portfolio: Level 3.

Recommended execution plan

```
Day 1: data download, raw views, profiling
Day 2: silver table, rejects, grain documentation
Day 3: dimension, fact table, daily zone metrics
Day 4: hourly demand, airport metrics,
month-over-month view
Day 5: validation checks, smoke test, EXPLAIN
review
Day 6: metric definitions, quality contract,
README
Day 7: backfill plan, incident runbook, business
summary, interview answer
```

Aktualizuj dokumenty w trakcie budowy. Nie odtwarzaj decyzji z pamięci na koncu.

Common weak submissions

Jeden plik all_queries.sql bez struktury.
Brak grain dla tabel gold.
Metryki policzone, ale niezdefiniowane.
Odrzucone rekordy znikaja bez rejected table.
README nie pozwala odtworzyc projektu.
Raport biznesowy tylko mowi, ze query dziala.
Brak walidacji unikalnosci i join quality.

Te problemy wygladaja jak lab. Capstone ma wygladac jak projekt.

■ Strong final narrative

Na koniec masz umieć powiedzieć:

Zbudowałem lokalny produkt danych na realnych plikach NYC Taxi.
DuckDB czyta Parquet i CSV w warstwie raw.
Silver oddziela poprawne przejazdy od rejected records.
Fact i dimension definiują model analityczny.
Gold tables odpowiadają na pytania o revenue, demand i airport trips.
Walidacje, smoke test i EXPLAIN zwiększają zaufanie do wyników.
README, backfill plan i runbook pokazują, jak projekt utrzymać.

To jest praktyczny sens Month 01 SQL Data Engineering.

■ Next step

Otworz brief projektu i przejdź po nim jak po specyfikacji wykonawczej.

```
02_month_01_capstone_project_instructions.md
```

Prezentacja daje kontekst i standard jakości.

Brief daje konkretne pliki, komendy, SQL outputy i acceptance criteria.